Design of a Kokkos-Based CVFEM Nodal Temperature Gradient Algorithm

1 Overview

This document describes the design of a stand-alone control-volume finite-element method (CVFEM) algorithm for computing the nodal temperature gradient in a Kokkos-based STK mesh application.

The implementation is intentionally independent of the existing MareNalu algorithm hierarchy. The design separates three responsibilities:

1. the physics algorithm, which manages mesh parts and orchestrates the calculation;
2. the CVFEM element implementation, which provides element-specific operations; and
3. the Kokkos execution model, which maps mesh buckets to Kokkos teams and elements to team threads.

The initial implementation supports the Tet4 SCV element. Hex8 SCV support will be added using the same interface.

2 Algorithm Object

A single `ComputeNodalGradTemperatureAlg` object represents one nodal-temperature-gradient algorithm.

The object contains references to the mesh and the fields required by the calculation:

- coordinates;
- temperature; and
- nodal temperature gradient.

The algorithm also contains a single vector,

`partVec_`,

containing all mesh parts to which the algorithm applies.

Thus, multiple element parts do not require multiple algorithm objects. When the algorithm is registered for another part, that part is appended to the existing `partVec_`.

3 Element-Type Dispatch

The algorithm supports multiple element topologies while retaining a single part vector.

The topology is therefore selected explicitly in `execute()`.

For example, the locally owned element selector is first constructed from all parts:

```
s_locally_owned_union =
locally_owned_part()
& selectUnion(partVec_);
```

The selector is then restricted to a particular topology:

```
s_tet4 =
s_locally_owned_union
& get_topology_root_part(TETRAHEDRON_4);
```

The Tet4 implementation is subsequently invoked through

```
execute_element<Tet4SCVElement>(s_tet4);
```

The same structure will be used for Hex8:

```
execute_element<Hex8SCVElement>(s_hex8);
```

Topology dispatch is deliberately visible in the main algorithm rather than being hidden inside the templated execution routine.

4 Nodal Output Initialization

The nodal gradient field is an output of the algorithm.

Before the element calculations begin, the field is initialized to zero.

The initialization selector is

```
s_all_nodes =
selectUnion(partVec_)
& selectField(gradTemperature_);
```

The locally-owned restriction is deliberately not included in this selector. A locally owned element can contribute to a shared node, so the nodal output must be initialized on all applicable nodes.

The output field is then made available to the device before the device calculation.

5 Read-Only and Read-Write Field Access

Field access follows the STK NGP read/write convention.

The coordinates and temperature fields are dependencies of the algorithm and are not modified:

```
auto coordinatesData =
coordinates_.data<
stk::mesh::ReadOnly,
stk::ngp::DeviceSpace>();

auto temperatureData =
temperature_.data<
stk::mesh::ReadOnly,
stk::ngp::DeviceSpace>();
```

The nodal temperature-gradient field is populated by the algorithm and is therefore accessed as read-write:

```
auto gradTemperatureData =
gradTemperature_.data<
stk::mesh::ReadWrite,
stk::ngp::DeviceSpace>();
```

The distinction between read-only and read-write fields establishes the access contract of the algorithm.

The algorithm is responsible for synchronization of the output field that it creates and populates. Synchronization of read-only input fields is outside the responsibility of this algorithm.

6 Bucket-Based Kokkos Execution

The execution model follows the STK NGP bucket-access pattern.

The algorithm first obtains the bucket IDs associated with the selected element topology:

```
stk::NgpVector<unsigned> bucketIds =
ngpMesh.get_bucket_ids(
stk::topology::ELEM_RANK,
selector);
```

A single Kokkos `TeamPolicy` launch is then used for all selected buckets.

The fundamental mapping is

$$\boxed{\text{one Kokkos team} = \text{one mesh bucket}}$$

rather than

$$\text{one Kokkos team} = \text{one element}.$$

Within each bucket, the elements are distributed among the team threads using

```
Kokkos::TeamThreadRange
```

This produces a nested execution structure:

$$\text{TeamPolicy} \longrightarrow \text{bucket} \longrightarrow \text{TeamThreadRange} \longrightarrow \text{element}.$$

This avoids a host-side loop over buckets followed by a separate device kernel launch for every bucket.

7 CVFEM Element Interface

Element-specific CVFEM operations are contained in dedicated element types.

The SCV and SCS element types are separate abstractions. An SCV element provides SCV operations, while an SCS element will provide SCS operations.

For the current Tet4 SCV implementation, the element provides:

- element coordinates;
- shape-function derivatives, `dndx`;
- SCV volumes; and
- the mapping from SCV integration point to its associated element node.

The element interface is represented by

`Tet4SCVElement`

and the corresponding Hex8 implementation will be

`Hex8SCVElement.`

8 Element Scratch Storage

Scratch storage is associated with the element type, but the algorithm allocates only the portions required by the operations that it performs.

The element therefore exposes independent scratch-size functions:

```
coordinates_scratch_size()
dndx_scratch_size()
scv_volume_scratch_size()
```

There is intentionally no single aggregate `scratch_size()` function.

For the current nodal temperature-gradient calculation, all three storage requirements are needed:

$$S_{\text{total}} = S_{\text{coordinates}} + S_{\text{dndx}} + S_{\text{scv}}.$$

A different algorithm that only requires coordinates and SCV volume can allocate

$$S_{\text{coordinates}} + S_{\text{scv}},$$

without allocating storage for `dndx`.

This allows the element type to describe all of its available data while allowing each algorithm to determine its actual scratch requirements.

9 Element Calculation

For each element, the calculation proceeds as follows.

1. Obtain the connected element nodes.
2. Gather nodal coordinates from the read-only coordinates field.
3. Gather nodal temperature from the read-only temperature field.
4. Compute the element shape-function derivatives.
5. Compute the SCV volumes.
6. Evaluate the temperature gradient at the SCV integration points.
7. Weight each contribution by its corresponding SCV volume.
8. Accumulate the resulting contribution into the nodal gradient field.

The element operations themselves are supplied by the element type:

```
Element::compute_dndx(...)
Element::compute_scv_volume(...)
Element::ip_node(...)
```

Consequently, the algorithm does not contain Tet4-specific or Hex8-specific geometric calculations.

10 Concurrent Nodal Accumulation

Multiple elements can contribute to the same nodal gradient. Because elements are processed concurrently on the device, the element contribution is accumulated using atomic operations:

```
Kokkos::atomic_add(...)
```

The element first forms its local contribution and then performs the atomic nodal accumulation.

This keeps the atomic operation outside the integration-point calculation and limits the number of atomic updates.

11 MPI Assembly

The device calculation produces the local contribution to the nodal gradient.

After the device calculation is complete, the output field is synchronized to the host and STK's parallel reduction is used to combine contributions to shared nodes:

```
stk::mesh::parallel_sum(
bulk_,
{&gradTemperature_});
```

The overall operation is therefore

initialize → device element calculation → host synchronization → MPI sum

12 Overall Execution Structure

The complete algorithm has the following conceptual structure:

```
execute()
|
+-- construct locally-owned element selector
|
+-- construct nodal output selector
|
+-- zero gradTemperature
|
+-- synchronize gradTemperature to device
|
+-- select Tet4 elements
|     |
|     +-- execute_element<Tet4SCVElement>()
|            |
|            +-- obtain bucket IDs
|            +-- one Kokkos team per bucket
|            +-- TeamThreadRange over elements
|            +-- element CVFEM operations
|            +-- atomic nodal accumulation
|
+-- select Hex8 elements
|     |
|     +-- execute_element<Hex8SCVElement>()
|            |
|            +-- same execution structure
|
+-- synchronize gradTemperature to host
|
+-- MPI parallel_sum
```

13 Design Principles

The implementation follows these principles:

1. One algorithm object can operate on multiple mesh parts.
2. A single `partVec_` is used for all supported element types.
3. Topology dispatch is explicit in `execute()`.
4. One Kokkos team represents one mesh bucket.
5. Elements within a bucket are processed with `TeamThreadRange`.
6. Element operations are separated from algorithm orchestration.

7. SCV and SCS element types remain distinct.
8. Scratch requirements are exposed independently for each element operation.
9. Read-only and read-write field access is explicitly declared.
10. The algorithm owns synchronization of the output field that it creates and populates.
11. Read-only input-field synchronization is not owned by this algorithm.
12. MPI assembly is performed after the device calculation.

14 Initial Implementation Scope

The initial implementation is intentionally limited to Tet4 SCV operations:

$$\text{Tet4 SCV} \quad \rightarrow \quad \{\texttt{dndx}, \texttt{scv_volume}\}\,.$$

Hex8 SCV support will subsequently implement the same interface.

SCS operations are outside the scope of the current nodal temperature-gradient calculation and will be introduced through a separate SCS element type.